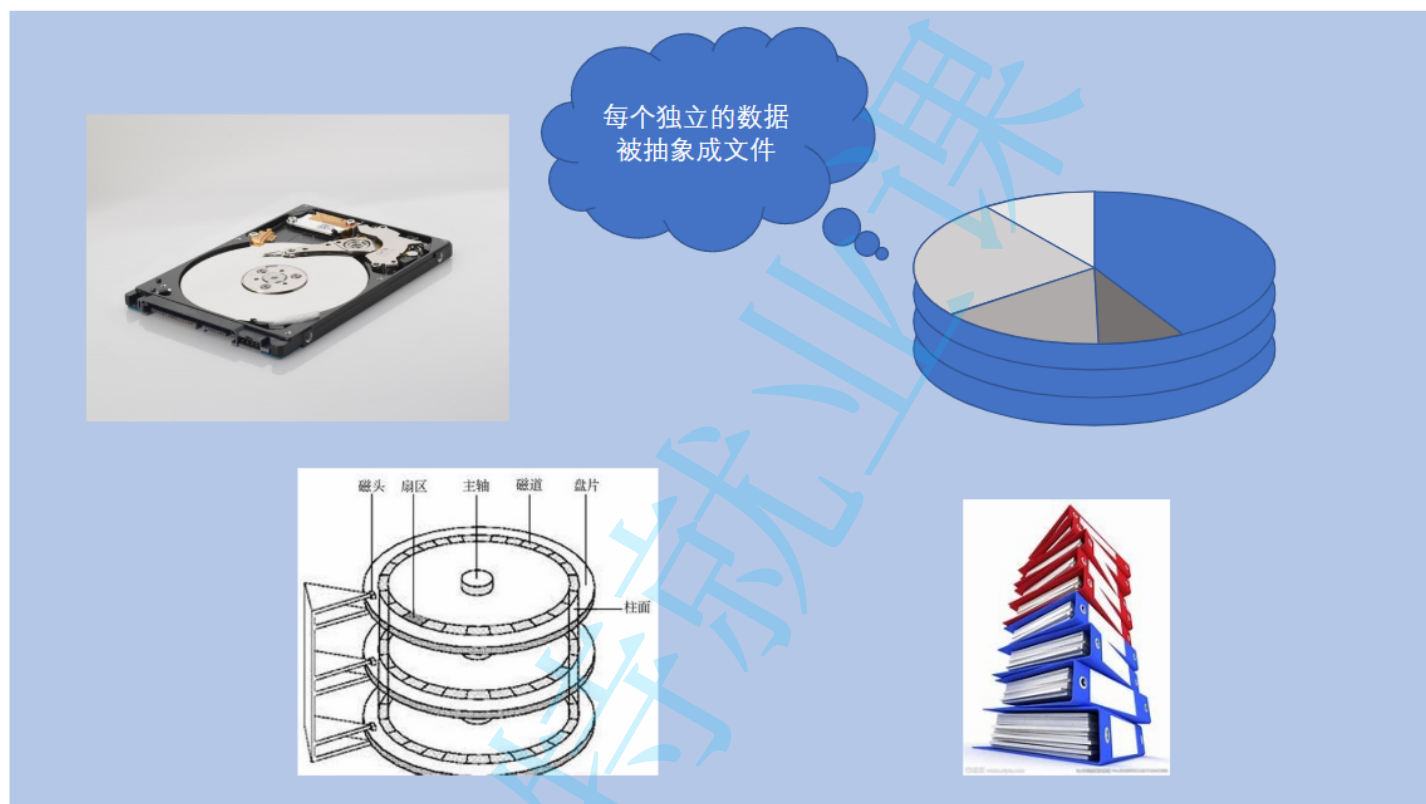


4. 文件操作和IO

认识文件

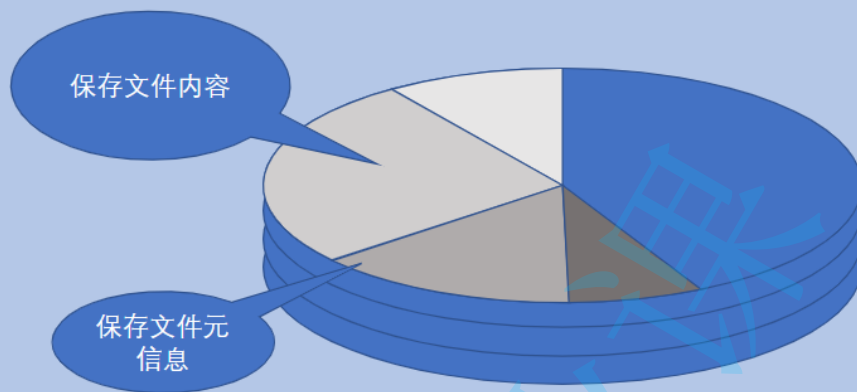
我们先来认识狭义上的文件(file)。针对硬盘这种持久化存储的I/O设备，当我们想要进行数据保存时，往往不是保存成一个整体，而是独立成一个个的单位进行保存，这个独立的单位就被抽象成文件的概念，就类似办公桌上的一份份真实的文件一般。



文件除了有数据内容之外，还有一部分信息，例如文件名、文件类型、文件大小等并不作为文件的数据而存在，我们把这部分信息可以视为文件的元信息。

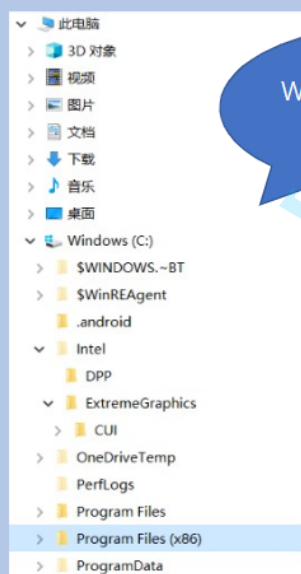
名称	修改日期	类型	大小
zh-CN	2019/3/19 19:41	文件夹	
PowerShellGet.psd1	2019/3/19 12:49	Windows PowerShell 数据文件	3 KB
PSGet.Format.ps1xml	2019/3/19 12:49	Windows PowerShell XML 文档	9 KB
PSGet.Resource.psd1	2019/3/19 12:49	Windows PowerShell 数据文件	79 KB
PSModule.psm1	2019/3/19 12:49	Windows PowerShell 脚本模块	572 KB

每个文件都有属于它自己的，不是内容的一些信息。



树型结构组织 和 目录

同时，随着文件越来越多，对文件的系统管理也被提上了日程，如何进行文件的组织呢，一种合乎自然的想法出现了，就是按照层级结构进行组织——也就是我们数据结构中学习过的树形结构。这样，一种专门用来存放管理信息的特殊文件诞生了，也就是我们平时所谓文件夹(folder)或者目录(directory)的概念。



Windows 上的树型结构组织

```
-- bin -> usr/bin
-- boot
-- config-3.10.0-1160.11.1.el7.x86_64
-- efi
-- EFI
-- centos
-- grub
-- splash.xpm.gz
-- grub2
-- device.map
-- fonts
-- unicode.pf2
-- grub.cfg
-- grub.cfg.1572966586.rpm.save
-- grubenv
-- i386-pc
-- acpi.mod
-- adler32.mod
-- affs.mod
-- afs.mod
-- ahci.mod
-- all_video.mod
-- aout.mod
-- archelp.mod
-- ata.mod
-- at_keyboard.mod
-- backtrace.mod
```

Linux 上的树型结构组织

Apowersoft	2021/3/5 15:36	文件夹
Bonjour	2021/3/5 15:37	文件夹
Common Files	2021/4/14 23:42	文件夹
Google	2021/5/8 10:29	文件夹
InstallShield Installation Information	2020/9/9 10:19	文件夹
Intel	2020/7/30 13:28	文件夹
Internet Explorer	2020/9/3 18:57	文件夹
Microsoft	2021/5/30 9:35	文件夹
Microsoft.NET	2019/12/20 17:04	文件夹
Mozilla Maintenance Service	2021/6/21 18:16	文件夹
MySQL	2020/11/25 18:22	文件夹
NetSarang	2020/9/9 10:19	文件夹
NVIDIA Corporation	2020/10/11 18:49	文件夹
QQMailPlugin	2020/9/1 11:26	文件夹
Tencent	2021/3/2 18:23	文件夹
Windows Defender	2021/5/17 0:14	文件夹
Windows Mail	2021/2/28 16:23	文件夹
Windows Media Player	2021/1/17 23:52	文件夹
Windows Multimedia Platform	2019/3/19 19:43	文件夹
Windows NT	2019/3/19 13:02	文件夹
Windows Photo Viewer	2021/1/17 23:52	文件夹
Windows Portable Devices	2019/3/19 19:43	文件夹
WindowsPowerShell	2019/3/19 12:52	文件夹

文件夹(folder)
目录(directory)
中保存的其实就是我们之前提到的关于文件的元信息。

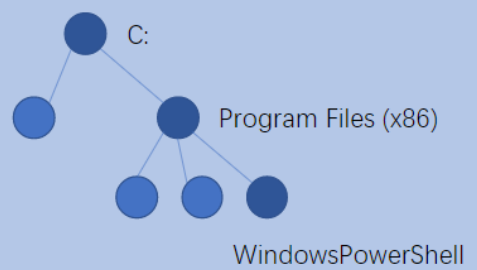
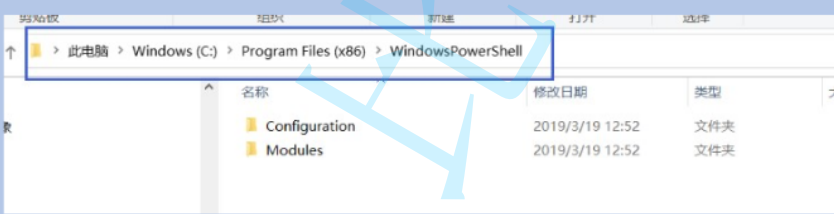
通过一个个的文件夹，我们可以将文件组织起来，更方便用户使用它。

逻辑上也更容易理解。



文件路径 (Path)

如何在文件系统中如何定位我们的一个唯一的文件就成为当前要解决的问题，但这难不倒计算机科学家，因为从树型结构的角度来看，树中的每个结点都可以被一条从根开始，一直到达的结点的路径所描述，而这种描述方式就被称为文件的绝对路径 (absolute path)。



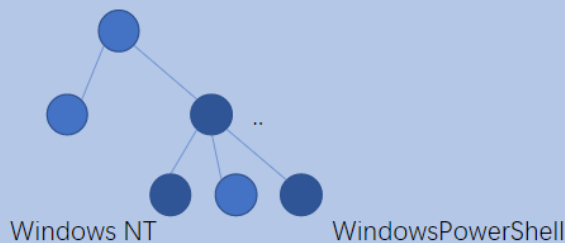
C:\Program Files (x86)\WindowsPowerShell

路径分隔符(Path Separator)

除了可以从根开始进行路径的描述，我们可以从任意结点出发，进行路径的描述，而这种描述方式就被称为相对路径 (relative path)，相对于当前所在结点的一条路径。

(C) 2019 Microsoft Corporation. 保留所有权利。

```
C:\Program Files (x86)\WindowsPowerShell>cd ..\Windows NT
```



从 WindowsPowerShell 出发
去往 Windows NT 的路径

· 代表当前结点
.. 代表父结点

其他知识

即使是普通文件，根据其保存数据的不同，也经常被分为不同的类型，我们一般简单的划分为文本文件和二进制文件，分别指代保存被字符集编码的文本和按照标准格式保存的非被字符集编码过的文件。

```
[root@VM-52-199-centos ~]# hexdump 我是中国人.txt
00000000 bde4 e5a0 bda5 bcef e68c 9188 98e6 e4af
00000010 adb8 9be5 e4bd baba 000a
00000019
```

```
[root@VM-52-199-centos ~]# cat 我是中国人.txt
你好，我是中国人
```

utf-8 编码	文本
0xe4bda0	你
0xe5a5bd	好
0xefbc8c	,
0xe68891	我
0xe698af	是
0xe4b8ad	中
0xe59bbd	国
0xe4baba	人
0x0a	\n

Name	Size	Offset	Description
Header	14 bytes		Windows Structure: BITMAPFILEHEADER
Signature	2 bytes	0000h	'BM'
FileSize	4 bytes	0002h	File size in bytes
reserved	4 bytes	0006h	unused (=0)
DataOffset	4 bytes	000Ah	Offset from beginning of file to the beginning of the bitmap data
InfoHeader	40 bytes		Windows Structure: BITMAPINFOHEADER
Size	4 bytes	000Eh	Size of InfoHeader =40
Width	4 bytes	0012h	Horizontal width of bitmap in pixels
Height	4 bytes	0016h	Vertical height of bitmap in pixels
Planes	2 bytes	001Ah	Number of Planes (=1)
Bits Per Pixel	2 bytes	001Ch	Bits per Pixel used to store palette entry information. This also identifies in an indirect way the number of possible colors. Possible values are: 1 = monochrome palette, NumColors = 1 4 = 4bit palitzized, NumColors = 16 8 = 8bit palitzized, NumColors = 256 16 = 16bit RGB, NumColors = 65536 24 = 24bit RGB, NumColors = 16777216
Compression	4 bytes	001Eh	Type of Compression 0 = BI_RGB, no color compression 1 = BI_RLE8 8bit RLE 2 = BI_RLE4 4bit RLE
ImageSize	4 bytes	0022h	(compressed) Size of the image data. It is valid to set this to 0 if the image is compressed.
XpixelsPerM	4 bytes	0026h	horizontal resolution: pixels per meter
YpixelsPerM	4 bytes	002Ah	vertical resolution: pixels per meter
Colors Used	4 bytes	002Eh	Number of actually used colors (0 = all colors)
Important Colors	4 bytes	0032h	Number of important colors (0 = all colors)

24 位位图的格式及数据

```
[root@VM-52-199-centos ~]# hexdump 我是个位图.bmp | less
```

```
00000000 4d42 1ee6 0004 0000 0000 0036 0000 0028
00000010 0000 012c 0000 012c 0000 0001 0018 0000
00000020 0000 1eb0 0004 0000 0000 0000 0000 0000
00000030 0000 0000 0000 1c24 24ed ed1c 1c24 24ed
00000040 ed1c 1c24 24ed ed1c 1c24 24ed ed1c 1c24
00000050 24ed ed1c 1c24 24ed ed1c 1c24 24ed ed1c
00000060 1c24 24ed ed1c 1c24 24ed ed1c 1c24 24ed
00000070 ed1c 1c24 24ed ed1c 1c24 24ed ed1c 1c24
00000080 24ed ed1c 1c24 24ed ed1c 1c24 24ed ed1c
00000090 1c24 24ed ed1c 1c24 24ed ed1c 1c24 24ed
000000a0 ed1c 1c24 24ed ed1c 1c24 24ed ed1c 1c24
000000b0 24ed ed1c 1c24 24ed ed1c 1c24 24ed ed1c
000000c0 1c24 24ed ed1c 1c24 24ed ed1c 1c24 24ed
```

Windows 操作系统上，会按照文件名中的后缀来确定文件类型以及该类型文件的默认打开程序。但这个习俗并不是通用的，在 OSX、Unix、Linux 等操作系统上，就没有这样的习惯，一般不对文件类型做如此精确地分类。

mpvis.DLL	2021/1/16 16:08	应用程序扩展	160 KB
setup_wm.exe	2021/1/16 16:08	应用程序	1,760 KB
wmlaunch.exe	2021/1/16 16:08	应用程序	71 KB
wmpconfig.exe	2019/11/22 9:42	应用程序	100 KB
wmplayer.exe	2019/11/22 9:42	应用程序	163 KB
WMPMediaSharing.dll	2021/1/16 16:08	应用程序扩展	99 KB
wmpnssci.dll	2021/1/16 16:08	应用程序扩展	422 KB
WMPNSSUI.dll	2019/3/19 19:42	应用程序扩展	18 KB
wmprph.exe	2021/1/16 16:08	应用程序	66 KB
wmpshare.exe	2019/11/22 9:42	应用程序	102 KB

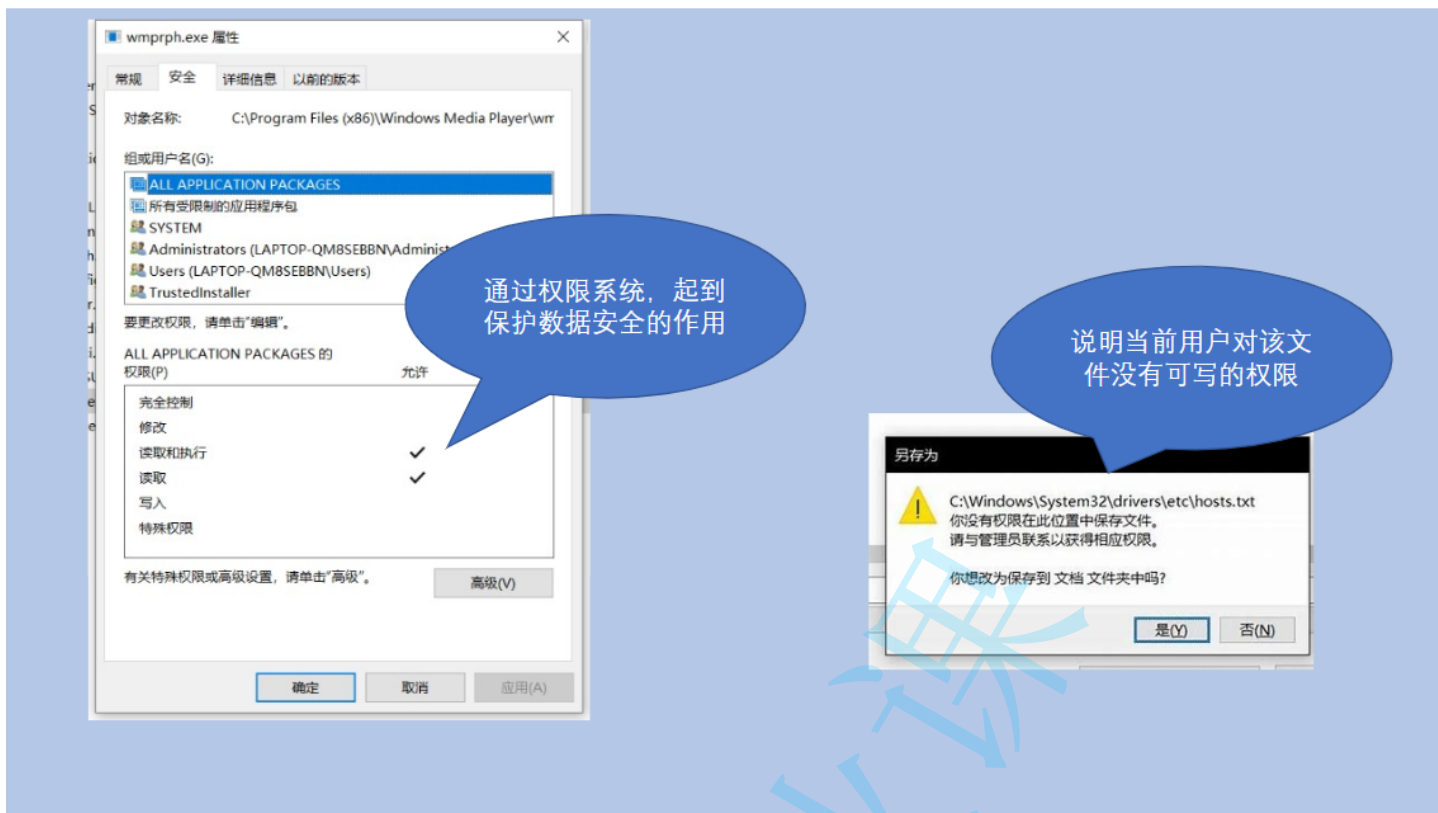
类似 exe、dll 这种，一般被称为文件的扩展名（extension）或者后缀（suffix）

不同的后缀往往代码不同的文件格式定义，起到不同的作用，例如：

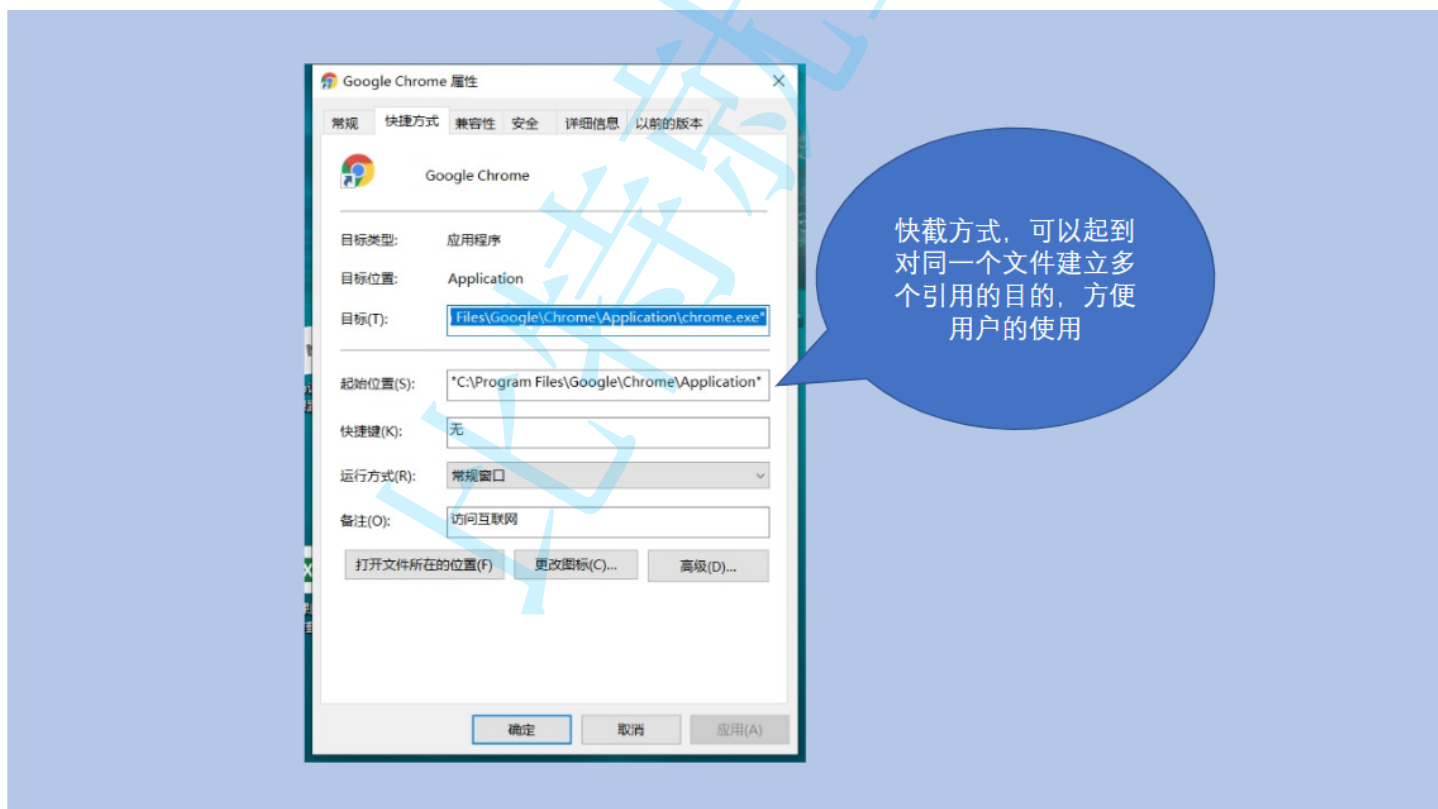
exe 在 Windows 中一般作为可执行程序来理解；

dll 在 Windows 中一般作为动态库（不包含入口的一组程序）来理解。

文件由于被操作系统进行了管理，所以根据不同的用户，会赋予用户不同的对待该文件的权限，一般地可以认为有可读、可写、可执行权限。



Windows 操作系统上，还有一类文件比较特殊，就是平时我们看到的快捷方式（shortcut），这种文件只是对真实文件的一种引用而已。其他操作系统上也有类似的概念，例如，软链接（soft link）等。



最后，很多操作系统为了实现接口的统一性，将所有的 I/O 设备都抽象成了文件的概念，使用这一理念最为知名的就是 Unix、Linux 操作系统 —— 万物皆文件。

Java 中操作文件

本节内容中，我们主要涉及文件的元信息、路径的操作，暂时不涉及关于文件中内容的读写操作。

Java 中通过 `java.io.File` 类来对一个文件（包括目录）进行抽象的描述。注意，有 File 对象，并不代表真实存在该文件。

File 概述

我们先来看看 `File` 类中的常见属性、构造方法和方法

属性

修饰符及类型	属性	说明
static String	pathSeparator	依赖于系统的路径分隔符，String 类型的表示
static char	pathSeparator	依赖于系统的路径分隔符，char 类型的表示

构造方法

签名	说明
File(File parent, String child)	根据父目录 + 孩子文件路径，创建一个新的 File 实例
File(String pathname)	根据文件路径创建一个新的 File 实例，路径可以是绝对路径或者相对路径
File(String parent, String child)	根据父目录 + 孩子文件路径，创建一个新的 File 实例，父目录用路径表示

方法

修饰符及返回值类型	方法签名	说明
String	getParent()	返回 File 对象的父目录文件路径
String	getName()	返回 File 对象的纯文件名称
String	getPath()	返回 File 对象的文件路径
String	getAbsolutePath()	返回 File 对象的绝对路径
String	getCanonicalPath()	返回 File 对象的修饰过的绝对路径
boolean	exists()	判断 File 对象描述的文件是否真实存在

boolean	isDirectory()	判断 File 对象代表的文件是否是一个目录
boolean	isFile()	判断 File 对象代表的文件是否是一个普通文件
boolean	createNewFile()	根据 File 对象，自动创建一个空文件。成功创建后返回 true
boolean	delete()	根据 File 对象，删除该文件。成功删除后返回 true
void	deleteOnExit()	根据 File 对象，标注文件将被删除，删除动作会到 JVM 运行结束时才会进行
String[]	list()	返回 File 对象代表的目录下的所有文件名
File[]	listFiles()	返回 File 对象代表的目录下的所有文件，以 File 对象表示
boolean	mkdir()	创建 File 对象代表的目录
boolean	mkdirs()	创建 File 对象代表的目录，如果必要，会创建中间目录
boolean	renameTo(File dest)	进行文件改名，也可以视为我们平时的剪切、粘贴操作
boolean	canRead()	判断用户是否对文件有可读权限
boolean	canWrite()	判断用户是否对文件有可写权限

代码示例

示例1

观察 get 系列的特点和差异

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File file = new File("../hello-world.txt"); // 并不要求该文件真实存在
7         System.out.println(file.getParent());
```



```
8      System.out.println(file.getName());
9      System.out.println(file.getPath());
10     System.out.println(file.getAbsolutePath());
11     System.out.println(file.getCanonicalPath());
12 }
13 }
```

运行结果

```
1 ..
2 hello-world.txt
3 ..\hello-world.txt
4 D:\代码练习\文件示例1\..\hello-world.txt
5 D:\代码练习\hello-world.txt
```

示例2

普通文件的创建、删除

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File file = new File("hello-world.txt"); // 要求该文件不存在，才能看
7         System.out.println(file.exists());
8         System.out.println(file.isDirectory());
9         System.out.println(file.isFile());
10        System.out.println(file.createNewFile());
11        System.out.println(file.exists());
12        System.out.println(file.isDirectory());
13        System.out.println(file.isFile());
14        System.out.println(file.createNewFile());
15    }
16 }
```

运行结果

```
1 false
2 false
3 false
4 true
```

```
5 true
6 false
7 true
8 false
```

示例3

普通文件的删除

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File file = new File("some-file.txt"); // 要求该文件不存在，才能看到
7         System.out.println(file.exists());
8         System.out.println(file.createNewFile());
9         System.out.println(file.exists());
10        System.out.println(file.delete());
11        System.out.println(file.exists());
12    }
13 }
```

运行结果

```
1 false
2 true
3 true
4 true
5 false
```

示例4

观察 deleteOnExit 的现象

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File file = new File("some-file.txt"); // 要求该文件不存在，才能看到
7         System.out.println(file.exists());
```

```
8      System.out.println(file.createNewFile());
9      System.out.println(file.exists());
10     file.deleteOnExit();
11     System.out.println(file.exists());
12 }
13 }
```

运行结果

```
1 false
2 true
3 true
4 true
```

程序运行结束后，文件还是被删除了

示例5

观察目录的创建

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File dir = new File("some-dir"); // 要求该目录不存在，才能看到相同的I
7         System.out.println(dir.isDirectory());
8         System.out.println(dir.isFile());
9         System.out.println(dir.mkdir());
10        System.out.println(dir.isDirectory());
11        System.out.println(dir.isFile());
12    }
13 }
```

运行结果

```
1 false
2 false
3 true
4 true
5 false
```

示例6

观察目录创建2

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File dir = new File("some-parent\\some-dir"); // some-parent 和 s
7         System.out.println(dir.isDirectory());
8         System.out.println(dir.isFile());
9         System.out.println(dir.mkdir());
10        System.out.println(dir.isDirectory());
11        System.out.println(dir.isFile());
12    }
13 }
```

运行结果

```
1 false
2 false
3 false
4 false
5 false
```

mkdir() 的时候，如果中间目录不存在，则无法创建成功; mkdirs() 可以解决这个问题。

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File dir = new File("some-parent\\some-dir"); // some-parent 和 s
7         System.out.println(dir.isDirectory());
8         System.out.println(dir.isFile());
9         System.out.println(dir.mkdirs());
10        System.out.println(dir.isDirectory());
11        System.out.println(dir.isFile());
12    }
13 }
```

运行结果

```
1 false
2 false
3 true
4 true
5 false
```

示例7

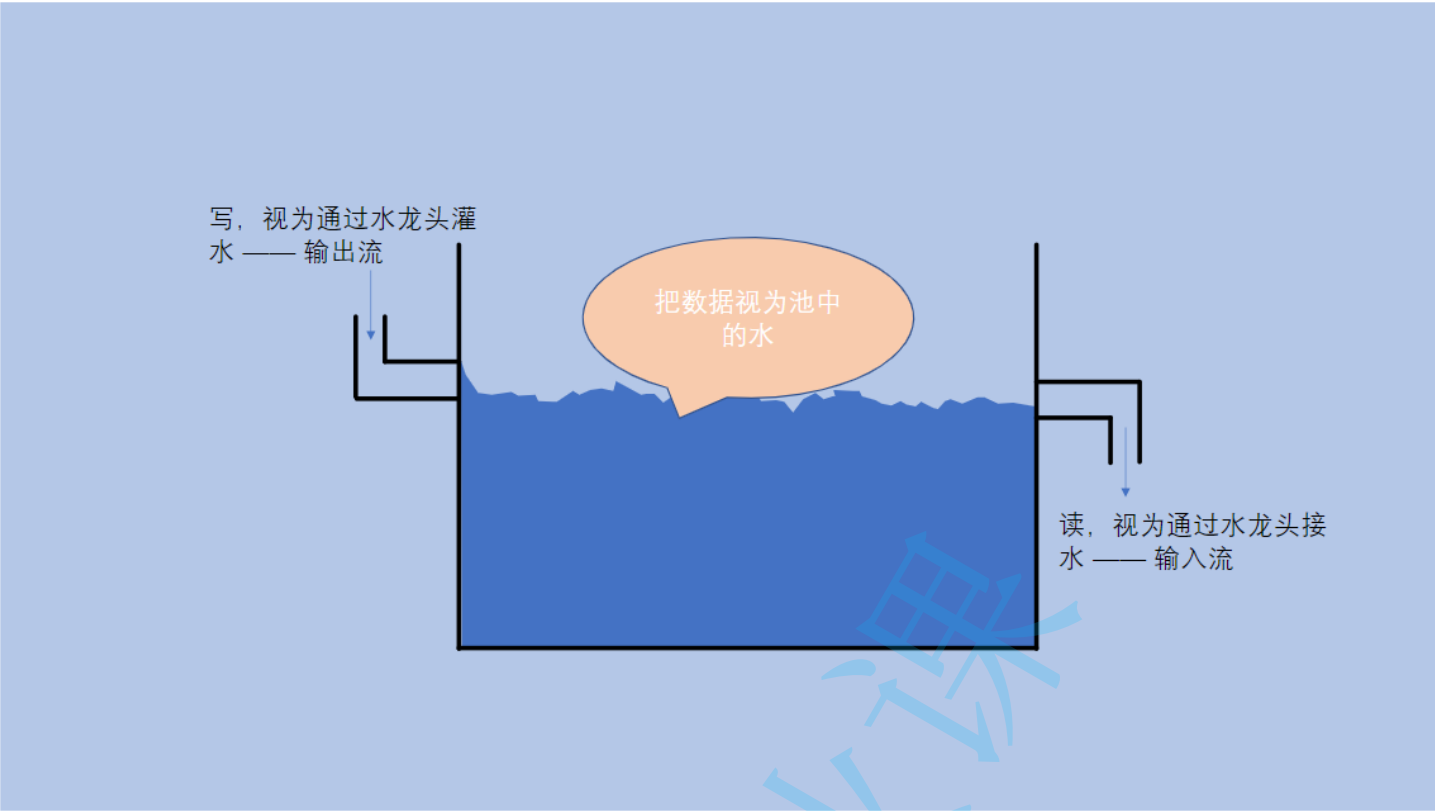
观察文件重命名

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         File file = new File("some-file.txt"); // 要求 some-file.txt 得存在
7         File dest = new File("dest.txt"); // 要求 dest.txt 不存在
8         System.out.println(file.exists());
9         System.out.println(dest.exists());
10        System.out.println(file.renameTo(dest));
11        System.out.println(file.exists());
12        System.out.println(dest.exists());
13    }
14 }
```

运行结果

```
1 true
2 false
3 true
4 false
5 true
```

文件内容的读写 —— 数据流



InputStream 概述

方法

修饰符及返回值类型	方法签名	说明
int	read()	读取一个字节的数据，返回 -1 代表已经完全读完了
int	read(byte[] b)	最多读取 b.length 字节的数据到 b 中，返回实际读到的数量；-1 代表以及读完了
int	read(byte[] b, int off, int len)	最多读取 len - off 字节的数据到 b 中，放在从 off 开始，返回实际读到的数量；-1 代表以及读完了
void	close()	关闭字节流

说明

InputStream 只是一个抽象类，要使用还需要具体的实现类。关于 InputStream 的实现类有很多，基本可以认为不同的输入设备都可以对应一个 InputStream 类，我们现在只关心从文件中读取，所以使用 **FileInputStream**

FileInputStream 概述

构造方法

签名	说明
FileInputStream(File file)	利用 File 构造文件输入流
FileInputStream(String name)	利用文件路径构造文件输入流

代码示例

示例1

将文件完全读完的两种方式。相比较而言，后一种的 IO 次数更少，性能更好。

```
1 import java.io.*;
2
3 // 需要先在项目目录下准备好一个 hello.txt 的文件，里面填充 "Hello" 的内容
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         try (InputStream is = new FileInputStream("hello.txt")) {
7             while (true) {
8                 int b = is.read();
9                 if (b == -1) {
10                     // 代表文件已经全部读完
11                     break;
12                 }
13
14                 System.out.printf("%c", b);
15             }
16         }
17     }
18 }
```

```
1 import java.io.*;
2
3 // 需要先在项目目录下准备好一个 hello.txt 的文件，里面填充 "Hello" 的内容
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         try (InputStream is = new FileInputStream("hello.txt")) {
7             byte[] buf = new byte[1024];
8             int len;
9
10             while (true) {
```

```

11         len = is.read(buf);
12         if (len == -1) {
13             // 代表文件已经全部读完
14             break;
15         }
16
17         for (int i = 0; i < len; i++) {
18             System.out.printf("%c", buf[i]);
19         }
20     }
21 }
22 }
23 }

```

示例2

这里我们把文件内容中填充中文看看，注意，写中文的时候使用 UTF-8 编码。hello.txt 中填写 "你好中国"

注意：这里我利用了这几个中文的 UTF-8 编码后长度刚好是 3 个字节和长度不超过 1024 字节的现状，但这种方式并不是通用的

```

1  import java.io.*;
2
3  // 需要先在项目目录下准备好一个 hello.txt 的文件，里面填充 "你好中国" 的内容
4  public class Main {
5      public static void main(String[] args) throws IOException {
6          try (InputStream is = new FileInputStream("hello.txt")) {
7              byte[] buf = new byte[1024];
8              int len;
9
10             while (true) {
11                 len = is.read(buf);
12                 if (len == -1) {
13                     // 代表文件已经全部读完
14                     break;
15                 }
16
17                 // 每次使用 3 字节进行 utf-8 解码，得到中文字符
18                 // 利用 String 中的构造方法完成
19                 // 这个方法了解下即可，不是通用的解决办法
20                 for (int i = 0; i < len; i += 3) {
21                     String s = new String(buf, i, 3, "UTF-8");
22                     System.out.printf("%s", s);
23                 }
24             }
25         }
26     }
27 }

```

```

25     }
26 }
27 }

```

利用 Scanner 进行字符读取

上述例子中，我们看到了对字符类型直接使用 `InputStream` 进行读取是非常麻烦且困难的，所以，我们使用一种我们之前比较熟悉的类来完成该工作，就是 `Scanner` 类。

构造方法	说明
<code>Scanner(InputStream is, String charset)</code>	使用 <code>charset</code> 字符集进行 <code>is</code> 的扫描读取

示例1

```

1 import java.io.*;
2 import java.util.*;
3
4 // 需要先在项目目录下准备好一个 hello.txt 的文件，里面填充 "你好中国" 的内容
5 public class Main {
6     public static void main(String[] args) throws IOException {
7         try (InputStream is = new FileInputStream("hello.txt")) {
8             try (Scanner scanner = new Scanner(is, "UTF-8")) {
9                 while (scanner.hasNext()) {
10                     String s = scanner.next();
11                     System.out.print(s);
12                 }
13             }
14         }
15     }
16 }

```

OutputStream 概述

方法

修饰符及返回值类型	方法签名	说明
<code>void</code>	<code>write(int b)</code>	写入要给字节的数据
<code>void</code>	<code>write(byte[] b)</code>	将 <code>b</code> 这个字符数组中的数据全部写入 <code>os</code> 中

int	write(byte[] b, int off, int len)	将 b 这个字符数组中从 off 开始的数据写入 os 中，一共写 len 个
void	close()	关闭字节流
void	flush()	重要：我们知道 I/O 的速度是很慢的，所以，大多的 OutputStream 为了减少设备操作的次数，在写数据的时候都会将数据先暂时写入内存的一个指定区域里，直到该区域满了或者其他指定条件时才真正将数据写入设备中，这个区域一般称为缓冲区。但造成一个结果，就是我们写的的数据，很可能会遗留一部分在缓冲区中。需要在最后或者合适的位置，调用 flush（刷新）操作，将数据刷到设备中。

说明

OutputStream 同样只是一个抽象类，要使用还需要具体的实现类。我们现在还是只关心写入文件中，所以使用 **FileOutputStream**

利用 OutputStreamWriter 进行字符写入

示例1

```
1 import java.io.*;
2
3 public class Main {
4     public static void main(String[] args) throws IOException {
5         try (OutputStream os = new FileOutputStream("output.txt")) {
6             os.write('H');
7             os.write('e');
8             os.write('l');
9             os.write('l');
10            os.write('o');
11            // 不要忘记 flush
12            os.flush();
13        }
14    }
15 }
```

```
1 import java.io.*;
2
3 public class Main {
4     public static void main(String[] args) throws IOException {
5         try (OutputStream os = new FileOutputStream("output.txt")) {
6             byte[] b = new byte[] {
7                 (byte)'G', (byte)'o', (byte)'o', (byte)'d'
8             };
9             os.write(b);
10
11             // 不要忘记 flush
12             os.flush();
13         }
14     }
15 }
```

```
1 import java.io.*;
2
3 public class Main {
4     public static void main(String[] args) throws IOException {
5         try (OutputStream os = new FileOutputStream("output.txt")) {
6             byte[] b = new byte[] {
7                 (byte)'G', (byte)'o', (byte)'o', (byte)'d', (byte)'B', (byte)'a'
8             };
9             os.write(b, 0, 4);
10
11             // 不要忘记 flush
12             os.flush();
13         }
14     }
15 }
```

```
1 import java.io.*;
2
3 public class Main {
4     public static void main(String[] args) throws IOException {
5         try (OutputStream os = new FileOutputStream("output.txt")) {
6             String s = "Nothing";
7             byte[] b = s.getBytes();
8             os.write(b);
9
10             // 不要忘记 flush
11             os.flush();
12         }
13     }
14 }
```

```
12     }
13 }
14 }
```

```
1 import java.io.*;
2
3 public class Main {
4     public static void main(String[] args) throws IOException {
5         try (OutputStream os = new FileOutputStream("output.txt")) {
6             String s = "你好中国";
7             byte[] b = s.getBytes("utf-8");
8             os.write(b);
9
10            // 不要忘记 flush
11            os.flush();
12        }
13    }
14 }
```

利用 PrintWriter 找到我们熟悉的方法

上述，我们其实已经完成输出工作，但总是有所不方便，我们接下来将 OutputStream 处理下，使用 PrintWriter 类来完成输出，因为

PrintWriter 类中提供了我们熟悉的 print/println/printf 方法

```
1 OutputStream os = ...;
2 OutputStreamWriter osWriter = new OutputStreamWriter(os, "utf-8"); // 告诉
3 PrintWriter writer = new PrintWriter(osWriter);
4
5 // 接下来我们就可以方便的使用 writer 提供的各种方法了
6 writer.print("Hello");
7 writer.println("你好");
8 writer.printf("%d: %s\n", 1, "没什么");
9
10 // 不要忘记 flush
11 writer.flush();
```

示例1

```
1 import java.io.*;
```

```

2
3 public class Main {
4     public static void main(String[] args) throws IOException {
5         try (OutputStream os = new FileOutputStream("output.txt")) {
6             try (OutputStreamWriter osWriter = new OutputStreamWriter(os, "UTF-8")) {
7                 try (PrintWriter writer = new PrintWriter(osWriter)) {
8                     writer.println("我是第一行");
9                     writer.print("我的第二行\r\n");
10                    writer.printf("%d: 我的第三行\r\n", 1 + 1);
11
12                    writer.flush();
13                }
14            }
15        }
16    }
17 }

```

小程序练习

我们学会了文件的基本操作 + 文件内容读写操作，接下来，我们实现一些小工具程序，来锻炼我们的能力。

示例1

扫描指定目录，并找到名称中包含指定字符的所有普通文件（不包含目录），并且后续询问用户是否要删除该文件

```

1 import java.io.*;
2 import java.util.*;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         Scanner scanner = new Scanner(System.in);
7         System.out.print("请输入要扫描的根目录（绝对路径 OR 相对路径）：");
8         String rootDirPath = scanner.next();
9
10        File rootDir = new File(rootDirPath);
11        if (!rootDir.isDirectory()) {
12            System.out.println("您输入的根目录不存在或者不是目录，退出");
13            return;
14        }
15
16        System.out.print("请输入要找出的文件名中的字符：");
17        String token = scanner.next();
18

```

```

19     List<File> result = new ArrayList<>();
20     // 因为文件系统是树形结构，所以我们使用深度优先遍历（递归）完成遍历
21     scanDir(rootDir, token, result);
22     System.out.println("共找到了符合条件的文件 " + result.size() + " 个，它们分别
23     for (File file : result) {
24         System.out.println(file.getCanonicalPath() + "      请问您是否要删除该文
25         String in = scanner.next();
26         if (in.toLowerCase().equals("y")) {
27             file.delete();
28         }
29     }
30 }
31
32 private static void scanDir(File rootDir, String token, List<File> result) {
33     File[] files = rootDir.listFiles();
34     if (files == null || files.length == 0) {
35         return;
36     }
37
38     for (File file : files) {
39         if (file.isDirectory()) {
40             scanDir(file, token, result);
41         } else {
42             if (file.getName().contains(token)) {
43                 result.add(file.getAbsolutePath());
44             }
45         }
46     }
47 }
48 }

```

示例2

进行普通文件的复制

```

1 import java.io.*;
2 import java.util.*;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         Scanner scanner = new Scanner(System.in);
7
8         System.out.print("请输入要复制的文件（绝对路径 OR 相对路径）：");
9         String sourcePath = scanner.next();
10        File sourceFile = new File(sourcePath);

```



```
11     if (!sourceFile.exists()) {
12         System.out.println("文件不存在，请确认路径是否正确");
13         return;
14     }
15
16     if (!sourceFile.isFile()) {
17         System.out.println("文件不是普通文件，请确认路径是否正确");
18         return;
19     }
20
21     System.out.print("请输入要复制到的目标路径（绝对路径 OR 相对路径）：");
22     String destPath = scanner.next();
23     File destFile = new File(destPath);
24     if (destFile.exists()) {
25         if (destFile.isDirectory()) {
26             System.out.println("目标路径已经存在，并且是一个目录，请确认路径是否正
27             return;
28         }
29
30         if (destFile.isFile()) {
31             System.out.println("目录路径已经存在，是否要进行覆盖？ y/n");
32             String ans = scanner.next();
33             if (!ans.toLowerCase().equals("y")) {
34                 System.out.println("停止复制");
35                 return;
36             }
37         }
38     }
39
40     try (InputStream is = new FileInputStream(sourceFile)) {
41         try (OutputStream os = new FileOutputStream(destFile)) {
42             byte[] buf = new byte[1024];
43             int len;
44
45             while (true) {
46                 len = is.read(buf);
47                 if (len == -1) {
48                     break;
49                 }
50
51                 os.write(buf, 0, len);
52             }
53
54             os.flush();
55         }
56     }
57
```

```
58     System.out.println("复制已完成");
59 }
60 }
```

示例3

扫描指定目录，并找到名称或者内容中包含指定字符的所有普通文件（不包含目录）

注意：我们现在的方案性能较差，所以尽量不要在太复杂的目录下或者大文件下实验

```
1  import java.io.*;
2  import java.util.*;
3
4  public class Main {
5      public static void main(String[] args) throws IOException {
6          Scanner scanner = new Scanner(System.in);
7          System.out.print("请输入要扫描的根目录（绝对路径 OR 相对路径）：");
8          String rootDirPath = scanner.next();
9
10         File rootDir = new File(rootDirPath);
11         if (!rootDir.isDirectory()) {
12             System.out.println("您输入的根目录不存在或者不是目录，退出");
13             return;
14         }
15
16         System.out.print("请输入要找出的文件名中的字符：");
17         String token = scanner.next();
18
19         List<File> result = new ArrayList<>();
20         // 因为文件系统是树形结构，所以我们使用深度优先遍历（递归）完成遍历
21         scanDirWithContent(rootDir, token, result);
22         System.out.println("共找到了符合条件的文件 " + result.size() + " 个，它们分别");
23         for (File file : result) {
24             System.out.println(file.getCanonicalPath());
25         }
26     }
27
28     private static void scanDirWithContent(File rootDir, String token, List<File>
29         File[] files = rootDir.listFiles();
30         if (files == null || files.length == 0) {
31             return;
32         }
33
34         for (File file : files) {
35             if (file.isDirectory()) {
36                 scanDirWithContent(file, token, result);
```

```

37         } else {
38             if (isContentContains(file, token)) {
39                 result.add(file.getAbsolutePath());
40             }
41         }
42     }
43 }
44
45 // 我们全部按照utf-8的字符文件来处理
46 private static boolean isContentContains(File file, String token) throws IOException {
47     StringBuilder sb = new StringBuilder();
48     try (InputStream is = new FileInputStream(file)) {
49         try (Scanner scanner = new Scanner(is, "UTF-8")) {
50             while (scanner.hasNextLine()) {
51                 sb.append(scanner.nextLine());
52                 sb.append("\r\n");
53             }
54         }
55     }
56
57     return sb.indexOf(token) != -1;
58 }
59 }

```

代码参考

如何按字节进行数据读

```

1  try (InputStream is = ...) {
2      byte[] buf = new byte[1024];
3      while (true) {
4          int n = is.read(buf);
5          if (n == -1) {
6              break;
7          }
8
9          // buf 的 [0, n) 表示读到的数据，按业务进行处理
10     }
11 }

```

如何按字节进行数据写

```

1 try (OutputStream os = ...) {
2     byte[] buf = new byte[1024];
3     while (/* 还有未完成的业务数据 */) {
4         // 将业务数据填入 buf 中, 长度为 n
5         int n = ...;
6         os.write(buf, 0, n);
7     }
8     os.flush(); // 进行数据刷新操作
9 }

```

如何按字符进行数据读

```

1 try (InputStream is = ...) {
2     try (Scanner scanner = new Scanner(is, "UTF-8")) {
3         while (scanner.hasNextLine()) {
4             String line = scanner.nextLine();
5
6             // 根据 line 做业务处理
7         }
8     }
9 }

```

如何按字符进行数据写

```

1 try (OutputStream os = ...) {
2     try (OutputStreamWriter osWriter = new OutputStreamWriter(os, "UTF-8")) {
3         try (PrintWriter writer = new PrintWriter(osWriter)) {
4             while (/* 还有未完成的业务数据 */) {
5                 writer.println(...);
6             }
7             writer.flush(); // 进行数据刷新操作
8         }
9     }
10 }

```